JavaScript Basics — για όσους ήδη ξέρουν Python

Πού τρέχει η JavaScript

Η Python συνήθως τρέχει σε ένα παράθυρο τερματικού. Η JS τρέχει σε ένα web browser και αλλάζει την HTML (δηλαδή τον κώδικα της ιστοσελίδας) σε πραγματικό χρόνο.

> i Γι' αυτό η JavaScript είναι η γλώσσα των ιστοσελίδων: ό,τι κουνιέται, αλλάζει ή απαντάει όταν πατάς κάτι σε μια σελίδα, συνήθως είναι JS.

Τερματισμός εντολών

Στην Python, μια εντολή ολοκληρώνεται όταν πατάμε “Enter” και μεταβαίνουμε στην κάτω γραμμή.

Στη JS, οι εντολές χωρίζονται μεταξύ τους με `;`

PYTHON

```python
age = 12
color = "red"
```

JAVASCRIPT

```javascript
let age = 12;
let color = "red";
```

Επειδή οι εντολές χωρίζονται με `;` και όχι με αλλαγή γραμμής, στη JS μπορούμε να γράψουμε και δύο εντολές στην ίδια γραμμή — αλλά είναι πιο ευανάγνωστο να μένει μία εντολή ανά γραμμή.

Οργάνωση εντολών

Για να ομαδοποιήσουμε εντολές στην Python (π.χ. τις εντολές που θέλουμε να επαναλαμβάνονται εντός ενός for loop) χρησιμοποιούμε indentation (enter + tab).

Στη JS, γράφουμε τον κώδικα εντός `{ }`

PYTHON

```python
if score > 10:
    score = 0
    print("You won!")
```

JAVASCRIPT

```javascript
if (score > 10) {
    score = 0;
    console.log("You won!");
}
```

> i Στη JS η συνθήκη μπαίνει πάντα μέσα σε παρενθέσεις `( )`, και δεν υπάρχει η άνω-κάτω τελεία `:` της Python. Η εσοχή δεν είναι υποχρεωτική — τη βάζουμε όμως για να διαβάζεται ο κώδικας.

Ορισμός μεταβλητών

Όταν ορίζουμε μια μεταβλητή στη JavaScript, πρέπει να δηλώνουμε αν σκοπεύουμε να μεταβάλουμε την τιμή της στη συνέχεια του προγράμματος.

Λέξη	Σημασία
const	Η τιμή της μεταβλητής δεν πρέπει να αλλάξει.
let	Επιτρέπονται νέες αναθέσεις.

JAVASCRIPT

```
const age = 12;
age = 13;        // ERROR
```

ΠΡΟΣΟΧΗ: Αυτό που απαγορεύεται είναι η νέα ανάθεση. Αν έχουμε μια λίστα, επιτρέπεται να αλλάξουμε το περιεχόμενο της, με μεθόδους επεξεργασίας λιστών.

JAVASCRIPT

```
const enemies = ["goblin", "orc"];
enemies.push("troll");    // επιτρέπεται: αλλάζουμε το περιεχόμενο
enemies = ["dragon"];     // ERROR: νέα ανάθεση
```

Στην Python δεν δηλώνουμε τίποτα: γράφουμε απλώς `age = 12`. Στη JS πρέπει πάντα να διαλέξουμε `const` ή `let`. Κανόνας: ξεκίνα με `const` και βάλε `let` μόνο αν όντως χρειάζεται να αλλάξει η τιμή.

Έξοδος

Για να εμφανίσουμε ένα μήνυμα στο τερματικό χρησιμοποιούμε την εντολή `console.log()` αντί για `print()`

PYTHON

```
print("hello world")
```

JAVASCRIPT

```
console.log("hello world");
```

Το τερματικό αλλάζει ανάλογα με την εφαρμογή. Αν τρέχουμε μια εφαρμογή στο browser, για να δούμε την έξοδο κάνουμε inspect και πηγαίνουμε στο tab console.

Λειτουργικό	Συντόμευση για inspect
Windows / Linux	`Ctrl + Shift + I`
Mac	`Cmd + Option + I`

String interpolation για “custom” prints

Περικλείουμε το string σε backticks `` `...` `` και εισάγουμε τις μεταβλητές σε `${..}`

PYTHON

```
error = "index error"
message = f"program exited with {error}"
```

JAVASCRIPT

```
const error = "index error";
const message = `program exited with
${error}`;
```

Και στις δύο γλώσσες το αποτέλεσμα είναι το ίδιο:

```
program exited with index error
```

ℹ Τα backticks `` ` `` είναι διαφορετικό σύμβολο από τα απλά εισαγωγικά. Σε ελληνικό πληκτρολόγιο βρίσκονται συνήθως στο πλήκτρο αριστερά από το `1`.

Έλεγχος ισότητας

Γενικά, γίνεται με `===` αντί για `==` όπως στην Python. Αντίστοιχα, το «διάφορο» γράφεται `!==` αντί για `!=`

Σημασία	Python	JavaScript
ίσο με	`==`	`===`
διάφορο από	`!=`	`!==`

Το `==` της JS κάνει αυτόματες μετατροπές τύπου (type casting) πριν συγκρίνει, που συχνά δεν είναι αυτές που περιμένουμε.

JAVASCRIPT

```
console.log("5" == 5);     // true  - μετατρέπει το "5" σε αριθμό
console.log("5" === 5);    // false - string και αριθμός δεν είναι το ίδιο
```

! **Κανόνας:** στη JavaScript χρησιμοποιούμε πάντα `===` και `!==`

fun fact: Την πρώτη φορά που είδα === στην JS γραμμένο από συνεργάτη μου, νόμιζα ότι ήταν bug και το άλλαξα σε == καταστρέφοντας το πρόγραμμα...

Συναρτήσεις

PYTHON

```
def add(a, b):
    return a + b
```

JAVASCRIPT

```
function add(a, b) {
    return a + b;
}
```

Η κλήση είναι ίδια και στις δύο γλώσσες:

PYTHON

```
result = add(3, 2)
```

JAVASCRIPT

```
const result = add(3, 2);
```

Arrow Functions (ΠΡΟΧΩΡΗΜΕΝΟ!)

Οι arrow functions είναι ένας πιο σύντομος τρόπος να γράψουμε μια συνάρτηση. Συνήθως τις αποθηκεύουμε σε μια `const` μεταβλητή, αλλά μπορούν να γραφτούν και κατευθείαν μέσα σε μια

άλλη εντολή — θα το δούμε στους πίνακες παρακάτω. Οι AI coding agents τις χρησιμοποιούν συνέχεια...

JAVASCRIPT

```javascript
const multiply = (a, b) => {
    return a * b;
};
```

ή αν έχουμε μόνο μια γραμμή κώδικα και δε χρειάζεται ομαδοποίηση:

JAVASCRIPT

```javascript
const multiply = (a, b) => a * b;
```

Σκεφτείτε την σαν τον ορισμό μιας συνάρτησης στα μαθηματικά.

Καλείται κανονικά ως συνάρτηση:

JAVASCRIPT

```javascript
const result = multiply(3, 2);
```

Δομή object (αντίστοιχη του python Dictionary)

PYTHON

```python
person = {"name": "Alex", "age": 50}

print(person["age"])
```

JAVASCRIPT

```javascript
const person = { name: "Alex", age: 50 };

console.log(person.age);
```

```
50
```

Δύο διαφορές: στη JS τα ονόματα των κλειδιών γράφονται **χωρίς εισαγωγικά**, και τα διαβάζουμε με **τελεία** (`person.age`) αντί για αγκύλες.

Επανάληψη σε πίνακες (for loops)

Το ακριβές αντίστοιχο του `for x in lista:` της Python είναι το `for ... of` της JS.

PYTHON

```python
enemies = ["goblin", "orc", "troll"]

for enemy in enemies:
    print(enemy)
```

JAVASCRIPT

```javascript
const enemies = ["goblin", "orc",
"troll"];

for (const enemy of enemies) {
    console.log(enemy);
}
```

```
goblin
orc
troll
```

Μέθοδοι πίνακα: forEach

Στη JS συναντάμε πολύ συχνά και έναν δεύτερο τρόπο: τη μέθοδο `forEach()`, στην οποία δίνουμε μια arrow function. Δεν είναι for loop — είναι μέθοδος του πίνακα — αλλά κάνει την ίδια δουλειά.

JAVASCRIPT

```javascript
const enemies = ["goblin", "orc", "troll"];

enemies.forEach(enemy => {
    console.log(enemy);
});
```

```
goblin
orc
troll
```

Εδώ βλέπουμε μια arrow function που **δεν** αποθηκεύεται σε μεταβλητή: γράφεται κατευθείαν μέσα στην `forEach`. Η μεταβλητή `enemy` παίρνει κάθε φορά το επόμενο στοιχείο του πίνακα, ακριβώς όπως στο `for ... of`.